

CMP-3222 – Machine Learning Technologies

Lab Sheet 1 (Week 2)

When: Monday 5th October 2026, 13:00–15:00.

Based on: Week 1 lectures (edshare links: L1.1 and L1.2)

There are three tasks to reinforce the Week 1 material:

- Task 1: Distance functions (timing + Numba)
- Task 2: scikit-learn preprocessing and pipelines
- Task 3: Implement a simple scikit-learn-compatible classifier

The code and data files for this lab are in the course GitHub repository and are also available on Moodle

GitHub: <https://github.com/TonyBagnall/COMP3222-2026>

If you need some practice with NumPy, there is a notebook labW1_Intro_to_Jupyter in the github project.

Task 1: Distance functions/Numba

Measuring distance/similarity is fundamental in many ML algorithms. This task demonstrates why native Python with loops can be slow, and how Numba helps. All these functions take two 1D NumPy arrays and return a float. You can assume they are the same length.

1. Euclidean distance ($O(n)$): Implement a function with a single loop (no input checks) that computes the sum of squared differences, then returns the square root.
2. Numba version: Duplicate the function (with a different name) and decorate with `@njit`. Remember to set `cache=True` (see lecture 1.2 slide 40).
3. Timing experiment: Use the provided pattern to time both versions over increasing n . Plot time vs n on log–log axes with matplotlib. You will have to make n big as these functions are very fast. There is an example in the repository if you need help (lab_1.py), but it's good to get to grips with matplotlib.

Hint 1: there are different ways to time experiments. The simplest is

```
import time
start = time.time()
some_function_call()
time_taken = time.time()-start
```

Some background for Part 1.

Euclidean distance is the square root of the sum of squared differences

$$d_e(a, b) = \sqrt{\sum_{i=1}^m (a_i - b_i)^2}$$

Part 1 and 2 ask you to implement this as two Python functions that satisfies the signature.

```
def euclidean_distance_python(x: np.ndarray, y: np.ndarray) -> float:
```

Use a single loop to do it. Do not do any input checks, just a single loop finding sum of squares.

Optional Extension: Do tasks 2 and 3 first, then return here if you have time

Dynamic Time Warping (DTW, $O(n^2)$): DTW is a distance measure specifically for time series (see background below). Implement the standard version shown in Algorithm 1. Re-run the timing experiment with and without Numba. Estimate the speed-up for Euclidean and DTW from using Numba.

DTW is a time series distance measure. It takes two 1D arrays (assumed to be time series) and returns a float, just as Euclidean. Its unique feature is that it can compensate for misalignment between two time series. You can see me and Chris (a research fellow) talking about it at PyData [here](#)

The basic $O(n^2)$ algorithm is this.

Algorithm 1: Dynamic Time Warping (DTW)

```

1 Input:  $x, y$  time series of length  $m$ 
   Output:  $C$ : Dynamic Time Warping
2 Let  $C$  be a  $(m + 1) \times (m + 1)$  matrix initialised with all values  $\infty$ .
    $C_{0,0} \leftarrow 0$ 
3 for  $i \leftarrow 1$  to  $m$  do
4     for  $j \leftarrow 1$  to  $m$  do
5          $d \leftarrow C_{i-1,j-1} + (x_i - y_j)^2$ 
6          $v \leftarrow C_{i-1,j} + (x_i - y_j)^2$ 
7          $h \leftarrow C_{i,j-1} + (x_i - y_j)^2$ 
8          $C_{i,j} \leftarrow \min(d, h, v)$ 
9 return  $C_{m,m}$ 
```

Rerun your timing experiment, generating plots of time against increasing n on a log scale (example with matplotlib in the file `lab_1.py`). Estimate the speed-up for both Euclidean and DTW from using Numba

Hint: quickly initialise the matrix C with infinities using NumPy:

```

import numpy as np
n = len(x)
C = np.full((n + 1, n + 1), np.inf, dtype=np.float64)
C[0, 0] = 0.0
```

Task 2: scikit-learn preprocessing and pipelines.

In Lecture 2 we described three scenarios that may require some preprocessing: categorical variables, missing data and the need to normalise. In this exercise you will see how to deal with each in scikit-learn.

- A. Categorical variables:** the file **one_hot.csv** on Moodle and in the repo (directory **data/lab1**) for the associated project contains an example dataset that contains three continuous variables, one categorical ordinal and one categorical nominal. The last column is the class labels.

A	B	C	D	E	F
0.164174	-0.69858	-0.09373	2	BLUE	0
-0.43765	0.790853	0.864004	1	GREEN	0
-0.243	0.492028	1.447888	0	GREEN	0
-0.09466	-1.76094	-0.10142	0	RED	0
-1.14796	0.538064	0.411778	0	RED	0
1.613798	-0.1517	1.863154	2	RED	0
0.380007	-0.1524	-0.86946	1	BLUE	0
-0.8129	1.125773	0.409948	2	RED	0
0.71547	-1.01944	-1.99631	0	GREEN	0
-0.4152	1.898125	0.79473	2	GREEN	0
-0.39444	0.585633	0.564692	2	GREEN	0
-1.63121	0.250332	-0.68014	1	GREEN	0
-2.37309	-0.38445	0.545239	2	GREEN	0

Load the data (pandas → NumPy): there are different ways to load this data, the easiest being with pandas, something like this

```
import pandas as pd
import numpy as np

path = "../data/one_hot.csv" # Or where ever you put it
df = pd.read_csv(path, header=None,
                 names=["x1", "x2", "x3", "rank", "colour", "y"])

X_cont = df[["x1", "x2", "x3"]].to_numpy(dtype=float) # (n, 3)
rank = df["rank"].to_numpy(dtype=int) # (n,)
colour = df["colour"].to_numpy(dtype=str) # (n,)
y = df["y"].to_numpy(dtype=int) # (n,)
```

Combine features (make a purely numeric feature matrix by appending rank as a float column): You can use `np.column_stack` to do this.

One-hot encode colour. Find out how to do this using `sklearn.preprocessing.OneHotEncoder`

Note `OneHotEncoder` expects 2D data, so you will need to reshape.

Sanity check: Test it with any classifier: Check that you can fit a classifier with your reformatted X

A good tutorial on handling categorical variables is here https://inria.github.io/scikit-learn-mooc/python_scripts/03_categorical_pipeline.html

B. Missing values. The file **missing.csv** contains data with missing values. These are indicated by a blank cell or by the text NaN. Easiest way to load the data is with pandas `pd.read_csv`. Load the data into memory, then explore scikit-learn methods for imputing missing values in **sklearn.impute**. Read the [API](#)

And use one, say [SimpleImputer](#), to estimate the missing values.

C. Scaling/standardisation

There are many operations in [sklearn.preprocessing](#).

StandardScaler is the normal way to do this. Explore using it to standardise the data that has been imputed.

D. Pipelines

Chain preprocessing and a model into a single estimator:

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("clf", KNeighborsClassifier()),
])
```

If you want to go further, look at using `ColumnTransformer` to handle the original `one_hot` data that mixes categorical and continuous data.

E. Train/test split.

Always evaluate on held-out data:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, shuffle=True
)

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

More on evaluation in Week 5. We will be explaining how these work in subsequent weeks. Now explore the classification algorithms available in scikit.

Task 3: Implementing your own classifier.

This is a small implementation exercise, but you need to follow the scikit-learn conventions so that your classifier works with other tools later in the module.

Following the last slide of lecture 2, implement a classifier that uses a simple decision rule on one feature to classify. It will only work on binary classification problems. It's purely an exercise to get you ready for the coursework and remind you of the topics of the lecture.

In the **constructor**, set the index of the feature.

In **fit**, find the average value of the feature for each class. Whichever value is lower will be the “less than” prediction. Set the threshold to the median of all the values of the feature.

In **predict**, use the selected feature and the learned threshold to generate a prediction for each case.

To clarify with an example, suppose this is your data

A	B	C	D
-0.63784	-1.09963	0.72688	0
-0.95687	-0.20331	0.713396	0
0.110266	-3.12633	-1.91651	0
1.000967	1.69653	0.62952	0
-0.9482	0.282593	-0.4967	0
1.163326	0.087972	1.446579	0
0.928831	1.56303	0.242051	0
0.839486	0.300472	0.182968	0
1.150775	0.757816	-0.73537	0
-1.04222	0.768152	0.095347	0
-0.21027	-0.00868	-1.61072	0
0.408118	0.048634	1.283588	0

In the constructor, the column of interest is set (default to 0). In fit, you separate the first column by the class value, and work out the mean for all of class 0 for the column and the mean of all of class 1. You can do this very compactly with NumPy. Then work out the median threshold with the built-in function. If the average of class 0 is smaller than the average of class 1, then the decision rule is **“if value < threshold, then class 0, else class 1”**. This rule is then applied in predict.

If you finish early, explore the range of classifiers in scikit-learn, we will be using them next week.